

Relatório de Atividade: Problema Two Sum

Amin Issam hammoud

Aluno do 3º Período de Sistemas de Informação
Faculdade ISULPAR

26 de maio de 2026

1 Desenvolvimento e Lógica Utilizada

O problema exige que encontremos dois índices em um array cujo valor contido some a um determinado objetivo (*target*). A abordagem inicial mais intuitiva seria a "Força Bruta", que consiste em utilizar dois laços de repetição (loops aninhados) para testar todas as combinações possíveis. No entanto, essa solução gera uma complexidade de tempo quadrática ($O(n^2)$), o que não atende ao desafio proposto na atividade.

Para desenvolver uma solução mais eficiente com complexidade menor que $O(n^2)$, utilizamos a estrutura de dados de Tabela de Dispersão (*Hash Map* / Dicionário em Python). A lógica desenvolvida baseia-se em percorrer o array verificando o "complemento" de cada elemento.

A lógica passo a passo funciona da seguinte forma:

1. Inicializamos um *Hash Map* vazio que servirá para guardar o valor do número como "chave" e o seu índice no array original como "valor".
2. Iteramos sobre o array *nums* uma única vez.
3. Para cada elemento visitado (onde temos o valor atual e o seu índice correspondente), calculamos o seu complemento: `complemento = target - valor atual`.
4. Verificamos se esse `complemento` já existe no nosso *Hash Map*.
 - Se existir, significa que no passado nós visitamos o número que somado ao atual resulta no *target*. Logo, retornamos o índice armazenado no *Hash Map* e o índice atual.
 - Se não existir, nós adicionamos o elemento atual e o seu índice ao *Hash Map* e continuamos para a próxima iteração.

Essa abordagem garante que nunca utilizemos o mesmo elemento duas vezes e encontra a resposta em uma única passagem (Single-Pass).

2 Análise de Complexidade

A solução apresentada atinge os requisitos de complexidade exigidos para o desafio de otimização da atividade.

2.1 Complexidade de Tempo: $O(n)$

Atravessamos a lista que possui n elementos exatamente uma vez. Dentro do loop, a verificação de existência no *Hash Map* (tabela de dispersão) tem uma complexidade de tempo de $O(1)$ no caso médio, assim como a inserção de novos valores. Sendo assim, o tempo total da execução cresce de forma diretamente proporcional ao tamanho do array de entrada, resultando em uma complexidade linear $O(n)$.

2.2 Complexidade de Espaço: $O(n)$

Para ganhar essa eficiência no tempo, sacrificamos um pouco de espaço na memória. O *Hash Map* precisa armazenar os elementos iterados. No pior dos casos (onde o par de números que resolve o problema são os dois últimos elementos do array), teremos que guardar até $n - 1$ elementos dentro do dicionário. Consequentemente, a memória extra alocada escala linearmente com o tamanho dos dados de entrada, resultando em uma complexidade espacial de $O(n)$.

3 Testes Realizados

Para validar o algoritmo implementado, o script de código foi submetido aos cenários exigidos pela atividade, operando perfeitamente e respeitando a regra de que sempre haverá apenas uma resposta válida.

- **Teste 1:** Array de entrada [2, 7, 11, 15] com target 9.
 - **Resultado:** Retornou [0, 1]. (*Aprovado*)
 - **Explicação:** Os valores nos índices 0 e 1 são 2 e 7. A soma $2 + 7 = 9$.
- **Teste 2:** Array de entrada [3, 2, 4] com target 6.
 - **Resultado:** Retornou [1, 2]. (*Aprovado*)
 - **Explicação:** Os valores nos índices 1 e 2 são 2 e 4. A soma $2 + 4 = 6$.
- **Teste 3:** Array de entrada [3, 3] com target 6.
 - **Resultado:** Retornou [0, 1]. (*Aprovado*)
 - **Explicação:** Os valores nos índices 0 e 1 são 3 e 3. A soma $3 + 3 = 6$.

Todos os casos contemplaram as restrições, lidando corretamente com os índices e prevenindo o reuso do mesmo elemento para resolver o objetivo.